

PROJECT REPORT

ATM Simulator Project

A beginner-level console software project built using Python to simulate basic banking transactions and PIN authentication.

Submitted By: Kislaya Kumar

Registration No. : 26BCE10063

Branch: Computer Science

Institution: VIT Bhopal

2. Introduction

The **ATM Simulator** is a Python command-line application that replicates core Automated Teller Machine operations. It offers users a safe, interactive interface to verify their security PIN, view account balances, deposit cash, and withdraw money in real time.

3. Problem Statement

Beginners learning computer programming need practical, real-world examples of how conditions, logic loops, and state management work together. This ATM project demonstrates how financial transaction logic and validation rules are implemented cleanly in software.

4. Functional Requirements

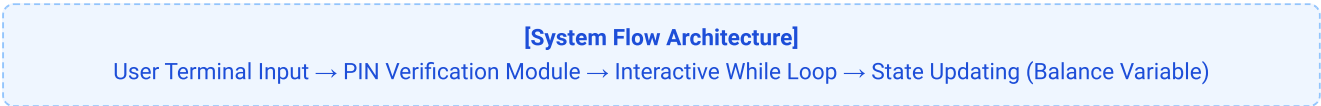
- **PIN Authentication:** Verifies user PIN (1234) before allowing access to menu features.
- **Balance Inquiry:** Displays the current account balance (Initial balance: Rs. 10,000).
- **Cash Withdrawal:** Deducts specified funds after checking for sufficient balance and valid amounts.
- **Cash Deposit:** Adds valid positive deposit amounts to the account balance.
- **Menu Loop & Exit:** Keeps showing menu choices until the user selects choice 4 to exit.

5. Non-Functional Requirements

- **Usability:** Simple, clear text prompts that guide the user step-by-step.
- **Reliability:** Prevents invalid withdrawal amounts (e.g., negative input or amount exceeding balance).
- **Speed:** Instantaneous terminal response for all user actions.

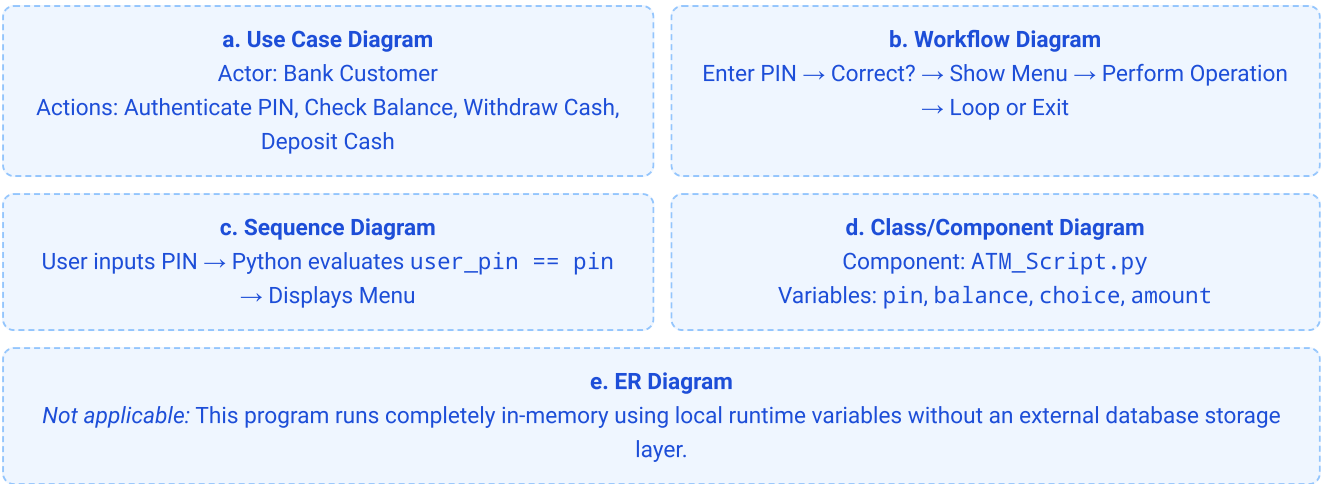
6. System Architecture

The system follows a standard single-script execution model operating in the Python interpreter memory:



7. Design Diagrams

Key diagrams illustrating the decision and execution flow of the ATM program:



8. Design Decisions & Rationale

Design Choice	Option Selected	Rationale
Programming Language	Python 3	Offers clean syntax, easy readability for beginners, and strong console I/O support.
Loop Control	<code>while True</code> Loop	Allows continuous menu choices until the user explicitly selects option 4 (Exit).
Data Handling	In-Memory Variables	Keeps code lightweight and simple for a basic programming project.

9. Implementation Details

Below is the complete, self-contained Python source code created for this project:

```
print("*****") print(" ATM SIMULATOR")
print("*****") pin = 1234 balance = 10000 user_pin = int(input("Enter your
PIN: ")) if user_pin == pin: print("\nLogin Successful!") while True: print("\n----- ATM MENU ----
---") print("1. Check Balance") print("2. Withdraw Money") print("3. Deposit Money") print("4.
Exit") choice = int(input("Enter your choice: ")) if choice == 1: print("\nYour balance is Rs.",
balance) elif choice == 2: amount = int(input("Enter amount to withdraw: ")) if amount <= 0:
print("Please enter a valid amount.") elif amount > balance: print("Insufficient balance.") else:
balance = balance - amount print("Please collect your cash.") print("Remaining balance: Rs.",
balance) elif choice == 3: amount = int(input("Enter amount to deposit: ")) if amount <= 0:
print("Please enter a valid amount.") else: balance = balance + amount print("Money deposited
successfully.") print("Updated balance: Rs.", balance) elif choice == 4: print("\nThank you for
using the ATM.") break else: print("Invalid choice. Please try again.") else: print("\nIncorrect
PIN.") print("Transaction cancelled.")
```

10. Screenshots / Results

Sample Terminal Output (Successful Login)

```
Enter your PIN: 1234
Login Successful!
Your balance is Rs. 10000
```

Sample Terminal Output (Withdrawal)

```
Enter amount to withdraw: 2000
Please collect your cash.
Remaining balance: Rs. 8000
```

11. Testing Approach

Test Case	Input Data	Expected Result	Status
PIN Validation	PIN = 1234	Login Successful, displays menu	PASS
Wrong PIN	PIN = 0000	"Incorrect PIN. Transaction cancelled."	PASS
Over-Withdrawal	Withdraw = 15000	"Insufficient balance."	PASS
Valid Deposit	Deposit = 3000	"Money deposited successfully. Updated balance: Rs. 13000"	PASS

12. Challenges Faced

- **Preventing Overdrafts:** Handled by checking `elif amount > balance:` before deducting funds.
- **Validating Inputs:** Added `if amount <= 0` conditions to prevent depositing or withdrawing zero or negative amounts.

13. Learnings & Key Takeaways

Through this project, I strengthened fundamental programming concepts including conditional branching (`if-elif-else`), infinite loops with break conditions (`while True`), integer input processing, and state variable updates in Python.

14. Future Enhancements

- Store account details and balances in a persistent database or file (e.g., MySQL or SQLite).
- Add support for multiple user accounts with unique PINs.
- Build a graphical user interface (GUI) using Tkinter or PyQt.

15. References

1. Official Python Documentation: <https://docs.python.org/3/>
2. W3Schools Python Tutorial: <https://www.w3schools.com/python/>